

Response to Reviewers

Dear Editor and Reviewer,

Thank you for the exceptionally thorough and constructive review of my manuscript, now titled *A Comparability Protocol for Benchmarking Relational Database Access Layers in Express.js*. This round (R8, the second major revision) responds to one comprehensive review, and I have addressed every point in it: all six Essential items, the eight numbered concerns, the minor labeling and abstract points, and the requests on novelty discipline, contribution separation, reproducibility, and structure.

I want to be explicit about scope at the outset: **the primary measurement matrix — the data, harness, pinned versions, raw per-cell records, and every previously reported primary number — is unchanged in this round.** Most changes are prose, label, and analysis-presentation revisions — reframing the contribution, rewriting the same-SQL result in non-causal terms, reweighting how the cross-engine transfer is characterized, softening the language of the low-replicate secondary experiments, and correcting a small number of category labels. Two points add new, clearly-scoped *supplementary* measurements that leave the primary matrix untouched: point 6.2 adds an independent write-state validation check, and point 6.4 adds a performance-conscious co-primary deep-fetch regime measured on its own harness. Below I answer each point in turn; every response quotes the current manuscript so that each change is verifiable, and I cite sections by name and supplement floats by number, since line numbers drift between builds.

Essential 1 — A single primary contribution, clearly stated and delimited

> The paper reads as several contributions at once (a methodology, an artifact, and an > empirical ranking); it should be reframed around one primary contribution, clearly > separated from the others.

Response. I agree, and I have subordinated the entire paper to **one** primary contribution — a *comparability protocol* — and explicitly distinguished it from both the artifact and the case study. The change is carried through the title (now *A Comparability Protocol for Benchmarking Relational Database Access Layers in Express.js*), the abstract, the Introduction, the Related Work positioning, the Discussion, the Conclusion, and the Highlights.

The Introduction states the contribution and its three mechanisms directly:

> "This paper's primary contribution is a **comparability protocol for access-layer > bench-**
marking: a set of controls that turn the vendor-benchmark genre — which reports > through-
put or latency numbers whose comparability is assumed — into a controlled > experiment, by
establishing three preconditions the genre otherwise leaves unmet."

The three preconditions are then named as **Semantic equivalence** ("Every layer must return *byte-identical* results ... before its timing is admitted"), **Strategy attribution** ("a *same-SQL control* bounds how much of an observed difference is the library's execution machinery versus the query strategy selected for it"), and **Operating-point separation** ("Capacity ..., tail latency under equal demand, and latency at matched utilization are measured and reported as *distinct* quantities rather than conflated at one arbitrary load point").

The same paragraph then separates the contribution from the two things it is easily confused with:

> "We distinguish this contribution from two things it is easily confused with. The > **artifact** — an open, reusable harness ... — *operationalizes* the protocol and is > released so it can be re-run, but is not itself the scientific claim ... The **empirical** > **case study** ... *demonstrates* the protocol and shows why each precondition is > load-bearing."

The Conclusion opens on the same footing ("This paper's contribution is a **comparability protocol** that makes access-layer benchmarking a controlled experiment rather than vendor-style number-reporting"), the abstract's Objective closes with "The protocol, not any ranking, is the contribution," and the Related Work Positioning now reads "the paper's contribution is the *comparability protocol* it applies." The first Highlight is now "A reusable comparability protocol turns access-layer benchmarks into experiments."

Essential 4 (points 2 & 3) — The same-SQL contrast is a compound intervention

> The same-SQL comparison is presented as if it isolated the library's machinery from the > query strategy, but it changes several things at once. Its construct validity as an > estimand, and the fact that it cannot attribute the residual to any single mechanism, > should be stated plainly.

Response. This was the most important scientific correction, and I have rewritten the same-SQL result so that it is described consistently as a *compound* contrast that *bounds rather than decomposes* the difference, and added a new main-text component diagram (Table 6, `tab:samesql_components`) that enumerates exactly what the intervention changes and what it does not.

The Results "Same-SQL control" subsection now opens:

> "The *same-SQL control* ... is a *compound* intervention: replacing each layer's > documentation-selected relational-loading path with the same hand-written SQL through its > raw facility changes the loading API, query strategy, SQL formulation, round-trip count, > statement preparation, and result hydration *together* (Table~6). ... It therefore > *bounds* the combined difference these components make; it does not attribute that > difference to any one of them, and in particular does not establish how much comes from > SQL formulation or query count."

The result itself is stated as a bound, not a decomposition:

> "The result this supports is that the documentation-selected paths differ much more than > the raw paths executing common SQL; it does *not* quantify how much of the original > difference comes from SQL formulation or query count, which change together with the API, > protocol, preparation, and hydration."

The new **Table 6** (`tab:samesql_components`) lists, component by component (Loading API, Query strategy, SQL formulation, Round-trip count, Statement preparation, Wire protocol, Result hydration), what the same-SQL control equalizes versus what still differs across layers, with the caption stating that "The residual difference it leaves therefore bounds their combined effect and cannot be attributed to any single one — in particular, not to SQL formulation or query count."

For the **construct validity of the treatment**, the Threats section now carries a dedicated paragraph describing the same-SQL control as "a compound contrast that *bounds*, rather than decomposes, how much of RQ1's spread comes from the documentation-selected loading strategy as opposed to the library machinery," and the introductory Results sentence anchors it as "a

compound contrast that bounds rather than decomposes the difference (construct validity: Section 6 [Threats])." A new supplement section, *Construct validity of the documentation-selected treatment* (Supplement Table S33), records per layer that the selected API is the library's canonical relations mechanism and disclaims representativeness: "The construct is therefore a defined, reproducible practitioner persona ... and is *not* a claim about performance-tuned expert usage or measured production frequency."

(The "bound / bounds" wording quoted above is the phrasing introduced by this fix; it was subsequently refined to a "standardized (same-SQL) contrast" in response to point 6.1, so the current manuscript reads "standardized contrast" wherever this section quotes "bound" — see the next response.)

Point 6.1 — The same-SQL result is a standardized contrast, not a "bound"

You observed that the default-path and raw-path measurements evaluate two *different* combinations of the factors that determine throughput, so — without assumptions about interaction effects — no ordering theorem makes the raw-path spread a bound on the intrinsic library effect, nor makes (default minus raw) a bound on the strategy contribution; raw mode may interact differently with each library and may bypass exactly the mechanisms one wishes to characterize. Following your strong recommendation, we did not attempt a formal proof but renamed the quantity throughout. Every "bound / bounds / bounded" applied to the same-SQL result is now a **standardized (same-SQL) contrast / diagnostic** — in the abstract, introduction, methodology, results (including the Table 6 caption and the same-SQL ratio table), threats, discussion, conclusion, related work, and the supplement; the only remaining "bound" mentions are explicit negations. Results now states your argument as the reason for the change:

> "Because raw mode changes several mechanisms at once and may interact differently with each library, this residual is a diagnostic contrast, not a bound on the intrinsic library effect — no ordering theorem holds without assumptions about interaction effects."

The compound-intervention framing (the contrast changes several mechanisms together; Table 6 enumerates the components it moves) is unchanged; only the inequality/ordering implication has been removed.

Point 6.2 — The semantic-equivalence gate now validates write state, not just a 2xx

You noted (a) that byte-identical responses over a finite probe set are strong output equivalence over the tested inputs, not a proof of full implementation equivalence, and (b) that a non-2xx check catches a failure like the MikroORM HTTP 500 but not a *silently incorrect* write. Following the option to strengthen (rather than rename), we added a companion check, **bench/verify-writes.mjs**, that validates post-write database state through an independent native-driver connection, for every adapter on both engines:

- the single-row insert wrote exactly the requested field values and generated identifier

(exactly one **posts** row added, no stray comments);

- the transactional write inserted the post and its five comments with the exact field values

(exactly one `posts` row and five `comments` rows added — one intended logical operation);

- a transaction whose last comment violates the author foreign key throws and leaves no partial state (atomic rollback).

All adapters pass on both engines, so no throughput number changes. Methodology now describes this and adds the honest finite-probe caveat for reads ("Byte-equality over a fixed probe set is a strong finite-probe output-equivalence check, not a proof that the implementations agree on every possible input"); the Discussion notes that a 2xx alone would not catch a silent write error, so database-state validation is the write path's correctness gate as byte-equality is the read path's. `REPRODUCE.md` and the artifact-reproducibility table (Supplement Table S31) include the new `node bench/verify-writes.mjs` command, which must print `ALL WRITES SEMANTICALLY CORRECT`.

Point 6.3 — The protocol is now specified independently of the case study

You noted that, although the paper's contribution is a reusable comparability protocol, its concrete definition was intertwined with the Express/access-layer case study. We added a Methodology subsection, "**The comparability protocol**" (Section referenced as `sec:protocol`), that states the protocol normatively and independently; the remaining subsections then instantiate it. It is given in the five parts you asked for:

- **Inputs** — the treatments, a representative workload (not necessarily a production trace), an

output semantics defining response equivalence, and an *operating-point definition* separating capacity, external demand, and utilization.

- **Mandatory stages, in order** — (1) correctness oracle, (2) treatment-definition rule, (3) strategy-control design, (4) capacity identification, (5) demand/utilization experiments.

- **Pass/fail (cell admission)** — a cell is admitted only if the oracle passes (equivalent non-mutating outputs and a correct post-write state, not merely a success status); non-equivalent output, an invalid or unsuccessful write, or a degenerate query plan (e.g. $N+1$) disqualifies it and it is excluded from timing.

- **Outputs and their interpretation** — throughput and tail latency per operating point, *descriptive* of the treatment-and-strategy, not a causal decomposition (the standardized contrast reports a residual); rankings are configuration- and version-specific, and only relative within-condition differences are meant to travel.

- **Applicability limits** — byte-identical comparison, our oracle here, is valid only when equivalent outputs are deterministic and canonically serializable; for unordered collections, floating-point results, timestamps, nondeterministic identifiers, or semantically equivalent but differently serialized responses, the oracle must instead be a semantic comparator (set equality, numeric tolerance, canonicalization, or field projection).

The Introduction contribution statement now points to this specification, so the protocol is presented with the abstraction its role as the primary contribution requires.

Point 6.4 — RQ1 narrowed to the *implementation strategy*, and a performance-conscious co-primary regime added

You made two requests. (**Part A, required**) that RQ1 be read as the performance of the *selected implementation strategy* rather than the "performance cost of the access layer," because the native driver's default is hand-authored SQL while an ORM's is whatever loader its documentation presents first — an asymmetry, with Drizzle borderline and Objection's MySQL default slower than its documented alternative. (**Part B, the stronger design you suggested**) that the study report two co-primary regimes — documentation-primary, and performance-conscious-but-semantically-equivalent under a predefined tuning budget — to separate "what the docs lead a developer to" from "what the technology can reasonably achieve." I have done both.

Part A. The GQM goal now reads "quantifying the performance of its *documentation-selected implementation strategy*" (previously "quantifying its performance cost"). The driver-versus-ORM asymmetry is stated explicitly in Study Design ("the native default is hand-authored SQL, the ORM default the library's relation loader"), and the construct-validity supplement already names the concrete cases (Drizzle's borderline builder-versus-relational-query choice; Objection's documented select-in default being the slower option on MySQL). I verified that no "performance cost / overhead of the access layer" phrasing remains anywhere in the manuscript; RQ1 in the Introduction already spoke of the "difference of the benchmarked implementations."

Part B. The deep/nested fetch is the only one of the five patterns that exposes a documented strategy choice (join versus select-in eager loading); on the other four each layer has a single documented path, so the two regimes coincide there by construction. For the deep fetch I added:

- A **tuning budget**, defined in Study Design: the performance-conscious regime takes the faster of

a layer's two *documented* deep-fetch loading strategies, admitted **only** where it is byte-identical to the documentation-primary output under the same `bench/verify.mjs` oracle (now extended to probe the alternative strategy at the three primary post ids on both engines). It never rewrites SQL, adds caching, or changes the schema.

- A **new harness** (`scripts/deepfetch-regimes.mjs`) that measures both regimes for the four

data-mapper ORMs on one footing (fixed post id, a warmup pass, then 25 timed repeats each), written to `results/deepfetch-regimes.json` and added to the reproducibility checksum manifest.

- A **new supplement float** (Table S34) and a Results paragraph reporting the two regimes side by side.

The finding reinforces, rather than overturns, the documentation-primary reading, and I say so plainly. For Sequelize and MikroORM the documentation-selected join is already the *faster* strategy, so the performance-conscious regime returns the identical configuration and the two coincide. The only cell that gains is Objection on MySQL, whose documented single-join alternative beats its select-in default (801 → 1,262 req/s, +58%) — the one case where the documentation leads to the slower option. Even there the performance-conscious ORM deep fetch remains below the native driver (1,262 versus the native `{mysql2}` baseline of 2,274 req/s on the same harness, 0.55×; Table S34 carries that reference row), so the deep-fetch gap is a property of the libraries' documented strategies, not a poor-default artifact. TypeORM's alternative errors on the ordered nested fetch and Objection's diverges from byte-identity on PostgreSQL; both are disclosed as non-drop-ins (a portability caveat), and there the performance-conscious regime falls back to documentation-primary by definition.

Point 6.5 — Capacity is now characterized for all five patterns, not only the deep fetch

You observed that the operating-point protocol is strongest for the deep fetch: the full five-pattern matrix is measured at a fixed 50 connections, but capacity sweeps existed only for the deep fetch, so cross-pattern p99 and spread comparisons could be affected by unknown relative utilization for the other four patterns. You offered two resolutions — demonstrate the separation across all five workloads, or explicitly scope it to the deep-fetch case. I took the first, and scoped honestly the one part I did not extend.

I swept every access layer over a connection ladder (1, 4, 8, 16, 32, 50, 100, 200) on **all five patterns** and both engines (`bench/scaling-patterns.mjs` → `results/scaling_patterns.json`), and for each layer computed its saturating throughput (the ladder maximum), its utilization at 50 connections (throughput at 50 / saturating), and its knee (the smallest connection count reaching 95% of saturation). The new **Supplement Table S35** reports, per pattern and engine, the median knee and the median and minimum utilization at 50 connections.

The result closes the gap: the knee lies at or below 50 connections for **every** pattern on both engines, so the fixed 50-connection point places all five patterns at high utilization of their own capacity. The relative utilization is therefore now **measured**, not assumed, and the RQ3 cross-pattern spread ranking compares the five at comparable (high) relative utilization. The binding constraint is the ten-connection pool shared by every pattern, which is why the knee clusters well below 50 connections regardless of a pattern's absolute throughput. Study Design and the RQ3 results now state this; capacity identification — a mandatory stage of the protocol — is thus demonstrated across all five patterns, not only the deep fetch.

I kept an explicit, honest scope on the two conditions I did **not** extend: the equal-utilization open-loop tail and the exploratory equal-compute check each require a per-layer saturating-throughput *target* and remain demonstrated on the deep fetch (the pattern with the largest layer spread). The manuscript now labels that as deep-fetch-only rather than implying it is general.

Essential 5 (point 4) — The n=7 rank correlations are given too much weight

> The cross-engine transfer rests heavily on Spearman coefficients computed over only seven > portable layers; concrete reversals and interaction magnitudes would be more informative.

Response. I reweighted the RQ2 treatment to lead with concrete rank reversals and with the layer×engine interaction magnitudes, and demoted the Spearman coefficients to coarse descriptive summaries. The RQ2 paragraph now opens:

> "The read ordering is stable across engines, but the aggregation and insert orderings > reverse at the top, so we characterize the transfer by these concrete rank reversals and by > the layer×engine interaction magnitudes rather than by a correlation coefficient computed > over only seven portable layers."

It then gives the reversals explicitly — on the insert, "`\{\}knex\{\}` leads on PostgreSQL (4,841 req/s) but drops to third on MySQL (1,670), where `\{\}drizzle\{\}` takes the lead (1,730);

Prisma falls furthest, from fourth on PostgreSQL (2,774) to last on MySQL (886 ...)"; on aggregation, "\{\}drizzle\{" leads on PostgreSQL (6,280) but falls to fourth on MySQL (3,520), where \{\}knex\{" leads (3,842)" — and reports the interaction in magnitude (per-layer PostgreSQL÷MySQL ratios: reads within ≈ 1.0 – $1.9\times$, insert $1.6\times$ to $3.2\times$, aggregation $1.4\times$ to $1.9\times$; Supplement Tables S27–S28). The coefficients now appear only "For completeness ... (descriptive only, and coarse at $n=7$)," closing with:

> "but the finding rests on the reversals and interaction spreads above, not on the > coefficients."

The abstract likewise now reports the read ordering once (Spearman $\rho \geq 0.86$) and leads its engine-dependence sentence with "the aggregation and insert orderings reverse: Prisma drops to last on the MySQL insert, and the per-layer engine advantage scatters 1.6 – $3.2\times$."

Essential 2 (point 5) — Narrow the five-pattern / synthetic-workload claims

> The Discussion generalizes from five probes on one synthetic schema to "ORM penalties" in > general; this over-reaches.

Response. I narrowed the generalization. The Discussion's Practical-guidance paragraph no longer speaks of ORM penalties as a category property; it locates the difference in the tested probes and labels the finding a benchmark observation to be re-measured:

> "This locates the difference among the five probes on a single synthetic schema, not across > relation-materializing workloads in general: one nested-fetch topology — the fan-out sweep > ... varies its breadth (0–500 children) but not its depth, its schema, or the mix of such > queries in a production workload — cannot establish a general distribution of ORM > penalties across relation-materializing traffic, so this is a benchmark observation to be > re-measured on the target workload, not an established law."

The subsequent sentence keeps the guidance conditional — "the tested thin paths (native driver and Knex) led here *for throughput and response-time p99*, but whether that generalizes to other implementations, schemas, or workloads is a hypothesis for broader study, not a category law or a general default" — and the Conclusion was scoped to match ("concentrated across the five tested patterns, not uniform").

Essential 3 (point 6) — Weaken the horizontal-scaling claim

> "Recoverable by horizontal scaling" overstates a bounded 1→4-worker check on one host with > no database-side saturation measurement, and it ignores the MySQL counter-case.

Response. I weakened the claim at every site. The unsupported "becomes database-bound" attribution was removed, the phrasing was changed to "may be addressed by additional application processes until another bottleneck is reached," scoped to the tested range, and the MySQL counter-case and the absence of any database-side saturation measurement are now stated explicitly. In the Results:

> "a layer's low per-process throughput may instead be addressed by additional application > processes until another bottleneck is reached — over the tested 1-to-4-worker range > aggregate throughput rose roughly in proportion to workers, though this run measures no > database-side saturation."

The supplement's *Multi-worker scaling* section carries the full caveat, including the MySQL counter-case:

> "This is a bounded check, not a demonstration of general scalability: it stops at four > workers, measures no database-side saturation (DB CPU, connections, or wait events), and on > MySQL horizontal scaling *widens* the gap — the native `\{\}mysql2\{\}` keeps scaling to > 8,883 req/s at four workers while Prisma reaches only 2,427 ... Within these limits a > layer's low per-process throughput may be addressed by additional application processes > until another bottleneck is reached."

The Discussion "Compute cost and horizontal scaling" paragraph makes the same disclaimer ("it measures no database-side saturation, on MySQL the native driver keeps scaling and the gap does not close, and scaling further would eventually reach another bottleneck ... this run does not locate"), and the Threats external-validity paragraph now says the multi-worker check "measures no database-side saturation, so scaling further would eventually reach a database or connection bottleneck it does not locate."

Essential 6 (point 7) — Cautious language for the low-replicate experiments

> Several secondary experiments run only 3–5 times but are reported with confirmatory verbs > ("confirms", "shows", "rules out").

Response. I matched the strength of the prose to the replicate count. Inferential verbs in the low-replicate secondaries became consistency statements, while deterministic checks and the primary-rigor claims kept their stronger wording. Specifically:

- **Equal-CPU (3 runs)** — "An exploratory equal-CPU check is consistent with no layer converting extra application cores into throughput" (Results), and the table caption now reads "the ranking does not appear to be an artifact of unequal core budgets."

- **Pool-size (3 runs)** — "a per-layer pool-size frontier ... is consistent with the ordering being preserved from a pool of 1 to 50 ... so the fixed pool does not appear to drive the ranking" (Threats), with the same softening in both supplement pool-size sections.

- **Transactional write (5 runs)** — now labelled exploratory in the supplement: "for a representative layer of each tier on PostgreSQL under default durability (median of five runs, exploratory)."

- **Open-loop coordinated-omission cross-run** — "is consistent with the tail ranking holding under that stricter model" (Threats).

- **Alternative eager-loading (5 runs)** — "argues against a mere default-strategy artifact" (Results, previously "rules out"), and "does not appear to be an artifact of a poor default" (Threats and the construct-validity supplement section).

By contrast, the deterministic checks (byte-identity, checksums) and the primary-rigor claims keep "confirms": the 25-run matrix and its paired permutation test, and the 25+10-run durability secondary whose relaxed run "confirms the cross-engine gap is not an artifact of either configuration" (Threats).

Essential-support (point 8) — Better discussion of the noisy, bimodal MySQL inserts

> The insert measurements are the noisiest and appear bimodal; the raw distributions should be > shown and discussed rather than reduced to a single CV.

Response. I brought the raw per-replicate insert distributions into the Results body and added a dedicated supplement treatment. The Measurement-stability subsection now states:

> "dispersion is larger only on the insert, where the MySQL cells are noisiest (up to 15.9%, > bimodal under group-commit batching; raw distributions in Supplement Figure~S1), so the > insert rank correlation is reported conservatively."

The supplement's *Dispersion on MySQL* section adds Figure S1, the raw per-replicate insert throughput behind the coefficients, and explains why a single CV is inadequate: "several cells are visibly bimodal or heavy-tailed within a cell, structure a single CV cannot convey, which is why the insert rank correlation and dispersion are reported conservatively."

Minor concerns

Abstract simplified (and the retired Prisma-version result removed from it). The abstract's Methods clause dropped the equal-compute-budget condition (an exploratory secondary), and its Conclusion dropped the retired five-core-Prisma parenthetical — that historical result now lives only in the Discussion and in the temporal external-validity threat, where it belongs. The structured abstract is 299 words, under the 300-word limit, and now reads more prominently on the core deep-fetch result.

"Vendor-independent" → "independently authored." The manuscript means authorship, not design, independence, so the study description and the coverage axis now use "independently authored" / "independent authorship," and the Study Design states it plainly: "Vendor independence means authorship only: no evaluated library's maintainers were involved, though the consequential author choices (selection, schema, pool size, tuning) remain and are disclosed." The prior-art table's "No (vendor)" column, which does concern authorship, is unchanged.

"orm-lightweight" merged into "orm." The interpretive, performance-implying label on an experimentally untested taxonomy was removed: Drizzle's Category column now reads "orm" in the six pattern/resource tables and both generators, and the methodology prose calls it "a query-builder-style ORM."

"documentation-selected" kept, and justified. I retained the term rather than renaming the estimand a second time, because it is a reproducible operational rule and the manuscript already disclaims any typical/best/recommended connotation. The methodology states it is "a reproducible selection heuristic, not evidence that the chosen API is performance-optimal or the most common production choice," and the new construct-validity treatment frames it as "a defined, reproducible practitioner persona — the documentation-following developer." A second rename would risk reopening the estimand discussion for no gain in precision.

Novelty discipline and contribution separation

The strongest novelty claim is scoped to the search. The Related Work Positioning now holds the claim to what a scoping (not systematic) search supports:

> "We therefore state the strongest novelty claim the search supports — that we did not identify a > prior study combining these properties in the documented search — rather than a claim of priority > the scoping search cannot establish."

No "first"/priority claim is made anywhere.

Novel experimental evidence is distinguished from the novel benchmark infrastructure. The Introduction's contribution paragraph now frames the case study as contributing novel evidence that is configuration-specific, distinct from — and less durable than — the protocol and harness:

> "The case study thus contributes novel experimental evidence ... but the resulting rankings are > configuration-specific and version-sensitive ... so the novel benchmark infrastructure (the protocol > and its harness), not any single ranking, is the more durable contribution."

The four coverage axes are named, and the HTTP-framework gap is placed beyond them. The Introduction now names the four axes explicitly — "access-layer taxonomy, engine, joint throughput/tail metrics, and independent authorship" — matching the prior-art table's four columns, and presents the separate client-observed/HTTP-framework gap as "Beyond these four axes, no study we found measures client-observed performance through an HTTP framework."

—

Reproducibility and structure

A reproducibility summary table. The supplement now carries an *Artifact reproducibility summary* (Supplement Table S31, `tab:reproducibility`): artifact and version, the persistent Zenodo identifier and concept DOI, the software and hardware requirements, the smoke-test and full-campaign run commands, the time and resources needed, and which results are regenerated automatically from the archived raw data ("Every table and figure, from the archived raw data; `\{results/checksums.sha256\}` verifies the 35 raw-data files"). The Data-availability section points to it, and `\{REPRODUCE.md\}` is the single runnable entry point.

Structure and streamlining. The load-bearing methodological detail was already relocated to the supplement in an earlier condensation (the *Statistical methods*, *Measurement details*, and *Benchmarking-pitfalls checklist* sections). This round added only a light streamlining: the duplicated "productivity/type-safety not measured" qualification in the Practical-guidance paragraph is now stated once, and a few redundant parenthetical "(Section ...)" cross-references in the densest sections (Results, Threats) were dropped where the same section was already cited nearby. No section was reordered, no float was moved, and no load-bearing caveat was removed — the caveats carry the Essential points above.

—

Closing

These revisions leave the manuscript making one clear scientific claim — a comparability protocol for access-layer benchmarking — demonstrated through a configuration-specific dual-engine case study whose rankings are disclosed as version-sensitive, with a supplement that serves as a complete audit trail. The manuscript remains under the journal's limit at **14,903 words** (IST rule) with a structured abstract of **297 words** (≤ 300), and, to reiterate, **the primary measurement matrix and every previously reported primary number are unchanged**; the only new measurements are the two clearly-scoped supplementary additions (the write-state

validation of point 6.2 and the co-primary deep-fetch regime of point 6.4), which leave the primary matrix untouched. The full replication package (harness, deterministic seed, all adapters, raw per-cell measurements, and the table-generating scripts) is permanently archived at Zenodo as release v1.7.0 (DOI 10.5281/zenodo.21461236), the version this revision describes.

I am grateful for the depth and precision of this review, which has materially sharpened the paper's central claim, and I look forward to your assessment.

Sincerely,

Mateusz Miotk Faculty of Mathematics, Physics and Informatics, University of Gdańsk